



Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey
Escuela de Ingeniería y Ciencias

Ingeniería en Ciencias de Datos y Matemáticas
Uso de álgebras modernas para seguridad y criptografía, MA2006B.604

SDK y ejemplos de interfaz – Gestor de Identidades y Verificación Documental

Documento técnico con flujos visuales de uso

Equipo 2:

Ana Lidia Hernández Díaz A00838643
Ana Paula García Valverde A01174572
María Fernanda Montoya López A01743214
Javier Rojas Orrante A01352213
Xavier Lago Hicks A01403177

Profesores: Raúl Gómez
Anas Wajid
Alberto F. Martínez

Proyecto: Firmas Solidarias – Tecnología Criptográfica para los Derechos Humanos
Socio Formador: Casa Monarca

Monterrey, Nuevo León. 12 de junio de 2026

Índice de Contenidos

1. Resumen Ejecutivo	1
2. Descripción General	2
2.1. ¿Qué es la aplicación?	2
2.2. ¿Para qué sirve?	2
2.3. ¿A quién va dirigida?	2
3. Arquitectura y Stack	3
3.1. Tipo de arquitectura	3
3.2. Componentes principales	3
3.3. Dependencias principales	3
3.4. Inicialización automática	3
4. Modelo de Datos y Seguridad	5
4.1. Entidades principales	5
4.2. Resumen funcional de tablas	5
4.3. Cifrado en reposo	5
4.4. Medidas de seguridad vigentes	5
5. Configuración e Instalación	7
5.1. Variables de entorno principales	7
5.2. Instalación local	7
5.3. Instalación con Docker	7
5.4. Despliegue en Render	7
6. Ejemplos de Interfaz	8
6.1. Ejemplo 1: Iniciar sesión como usuario no criptográfico	8
6.2. Ejemplo 2: Iniciar sesión como usuario criptográfico	8
6.3. Ejemplo 3: Acceso demo y usuarios disponibles	9
6.4. Ejemplo 4: Crear un usuario desde administración	9
6.5. Ejemplo 5: Auto-registro externo	10
6.6. Ejemplo 6: Activar un usuario pendiente	11
6.7. Ejemplo 7: Revocar un usuario	11
6.8. Ejemplo 8: Cambiar el rol de un usuario	12
6.9. Ejemplo 9: Reactivar o restaurar acceso	12
6.10. Ejemplo 10: Descargar credenciales criptográficas	13
6.11. Ejemplo 11: Registrar un beneficiario	13
6.12. Ejemplo 12: Actualizar el estado de un beneficiario	14
6.13. Ejemplo 13: Exportar reportes PDF	15
6.14. Ejemplo 14: Desbloquear una cuenta bloqueada	16
6.15. Ejemplo 15: Activar el administrador espejo	16
6.16. Ejemplo 16: Consultar la auditoría	17
6.17. Ejemplo 17: Firmar un documento	18
6.18. Ejemplo 18: Firma masiva de documentos	18
6.19. Ejemplo 20: Verificar un documento por folio	19
6.20. Ejemplo 21: Verificar un documento por archivo	20
7. Referencias Técnicas	21

Índice de Tablas

Table 1	Resumen de módulos vigentes del sistema	1
Table 2	Perfiles de uso del sistema	2
Table 3	Componentes principales del backend	3
Table 4	Dependencias del proyecto	3
Table 5	Resumen del modelo ORM actual	5
Table 6	Variables de entorno más relevantes	7
Table 7	Usuarios demo y método de acceso vigente según la implementación actual	9
Table 8	Reportes PDF exportables desde la aplicación	15
Table 9	Reglas funcionales del módulo documental	18

1. Resumen Ejecutivo

El sistema actual de Casa Monarca ya no es solamente un gestor de identidades. La versión vigente integra gestión de usuarios, autenticación con archivos criptográficos, firma documental, verificación pública, registro de beneficiarios, auditoría, reportes y notificaciones en una sola aplicación web desarrollada con FastAPI.

Este documento integra la parte técnica del sistema y una sección de ejemplos de interfaz alineados al comportamiento real del código. El objetivo no es replicar un manual completo, sino documentar el SDK y mostrar los flujos visuales clave de la aplicación.

Módulo	Estado	Alcance actual
Identidades	Activo	Alta, activación, revocación, cambio de rol, expiración, desbloqueo y recuperación por administrador espejo.
PKI	Activo	Emisión de llaves RSA-2048, certificados X.509 y reemisión de credenciales.
Documentos	Activo	Firma individual, firma masiva, cofirmas, revocación, verificación pública y exportación CSV.
Beneficiarios	Activo	CRUD operativo cifrado por áreas institucionales.
Auditoría y reportes	Activo	Bitácora de eventos y exportación PDF.
Notificaciones	Activo	Alertas persistentes y WebSocket en tiempo real.

Table 1: Resumen de módulos vigentes del sistema

Base documental del SDK El contenido técnico de este documento está alineado con la implementación vigente de `app/main.py`, `app/services.py`, `app/models.py`, `app/config.py`, `app/db.py`, `app/crypto.py` y `pyproject.toml`.

2. Descripción General

2.1. ¿Qué es la aplicación?

Es una plataforma web interna para Casa Monarca que administra identidades institucionales y controla accesos sensibles mediante autenticación tradicional y autenticación criptográfica.

El sistema distingue entre:

- usuarios que ingresan con correo o identificador y contraseña;
- usuarios con rol privilegiado que deben autenticarse con `private_key.pem`, `certificate.pem` y contraseña;
- terceros externos que no inician sesión, pero pueden verificar documentos emitidos por la organización.

2.2. ¿Para qué sirve?

La plataforma resuelve necesidades operativas y de seguridad como:

- gestión de usuarios;
- control RBAC;
- emisión de certificados X.509;
- autenticación criptográfica;
- firma y verificación documental;
- seguimiento de beneficiarios;
- generación de reportes;
- recuperación de emergencia;
- trazabilidad y alertas.

2.3. ¿A quién va dirigida?

Perfil	Método de acceso	Uso principal
Administrador	Archivos criptográficos	Administra usuarios, certificados, documentos, auditoría y recuperación.
Coordinador	Archivos criptográficos	Gestiona operaciones de su área y firma documentos autorizados.
Operativo	Correo + contraseña	Consulta y opera procesos internos autorizados.
Voluntario	Correo + contraseña	Acceso limitado de consulta y operación básica.
Tercero externo	Sin login	Valida documentos emitidos por Casa Monarca.

Table 2: Perfiles de uso del sistema

3. Arquitectura y Stack

3.1. Tipo de arquitectura

La implementación vigente es una aplicación monolítica en capas:

- presentación HTML desde FastAPI;
- lógica de negocio en `app/services.py`;
- persistencia con SQLAlchemy ORM;
- criptografía con `cryptography`;
- notificaciones push por WebSocket.

3.2. Componentes principales

Componente	Responsabilidad
<code>app/main.py</code>	Rutas HTML, API JSON, WebSocket, sesiones y renderizado.
<code>app/services.py</code>	Usuarios, permisos, certificados, documentos, beneficiarios, auditoría y notificaciones.
<code>app/models.py</code>	Modelo ORM completo del sistema.
<code>app/crypto.py</code>	Cifrado en reposo e índices ciegos HMAC.
<code>app/config.py</code>	Variables de entorno y validaciones de configuración.
<code>app/db.py</code>	Engine y sesiones de base de datos.

Table 3: Componentes principales del backend

3.3. Dependencias principales

Paquete	Versión mínima	Uso
<code>fastapi</code>	≥ 0.115.0	Framework web.
<code>sqlalchemy</code>	≥ 2.0.35	ORM y base de datos.
<code>cryptography</code>	≥ 43.0.0	RSA, X.509, Fernet y firmas.
<code>itsdangerous</code>	≥ 2.2.0	Firmado de sesiones.
<code>uvicorn[standard]</code>	≥ 0.30.0	Servidor ASGI.
<code>python-multipart</code>	≥ 0.0.9	Carga de archivos en formularios.
<code>psycopg[binary]</code>	≥ 3.2.0	Driver PostgreSQL.
<code>pydantic-settings</code>	≥ 2.5.2	Configuración tipada.
<code>fpdf2</code>	≥ 2.7.0	Reportes PDF.

Table 4: Dependencias del proyecto

3.4. Inicialización automática

Al arrancar, la app:

- crea tablas si no existen;

- si `SEED_DEMO_DATA=true`, siembra usuarios y beneficiarios demo;
- sincroniza roles y permisos;
- genera o refresca material criptográfico administrado;
- prepara el esquema dinámicamente con `SchemaService`.

4. Modelo de Datos y Seguridad

4.1. Entidades principales

- roles
- users
- permissions
- role_permissions
- audit_logs
- system_secrets
- notifications
- beneficiarios
- documents
- document_signatures
- document_required_signers
- document_verifications

4.2. Resumen funcional de tablas

Tabla	Función
users	Usuarios, estados, rol, certificado, llave y bloqueo de acceso.
documents	Metadatos del documento firmado: folio, hash, firma, estado y vigencia.
document_signatures	Cada firma individual en documentos con multifirma.
document_required_signers	Lista de firmantes requeridos y orden de firma.
document_verifications	Trazabilidad de verificaciones públicas.
beneficiarios	Personas atendidas por Casa Monarca con datos cifrados.
notifications	Alertas internas y estatus de lectura.
audit_logs	Registro de acciones sensibles con resultado e IP.

Table 5: Resumen del modelo ORM actual

4.3. Cifrado en reposo

Los campos sensibles usan:

- EncryptedText para texto cifrado;
- EncryptedJSON para JSON cifrado;
- Fernet para cifrado simétrico;
- HMAC-SHA256 para digests de búsqueda ciega.

4.4. Medidas de seguridad vigentes

- Contraseñas con PBKDF2-HMAC-SHA256 y 120,000 iteraciones.
- Certificados X.509 para ADMIN y COORDINADOR.
- Llaves RSA de 2048 bits.
- Firmas documentales con RSA-PSS-SHA256.
- Cookies httponly, samesite=lax y seguras en producción.

- Bloqueo automático por intentos fallidos.
- Auditoría con `ip_address` y `user_agent`.

5. Configuración e Instalación

5.1. Variables de entorno principales

Variable	Default	Uso
ENVIRONMENT	development	Controla el modo de ejecución.
DATABASE_URL	SQLite local	Conexión de base de datos.
DATABASE_ENCRYPTION_KEY	vacío	Clave estable de cifrado en reposo.
SESSION_SECRET	valor demo	Firma de sesiones web.
SEED_DEMO_DATA	true	Carga datos demo.
ALLOW_DEMO_ADMIN_BYPASS	derivado de demo	Activa el acceso admin/admin en modo demo.
BOOTSTRAP_ADMIN_EMAIL	vacío	Administrador inicial sin demo.
BOOTSTRAP_ADMIN_PASSWORD	vacío	Contraseña del administrador inicial.

Table 6: Variables de entorno más relevantes

5.2. Instalación local

1. Crear entorno virtual con `python -m venv venv`.
2. Activarlo con `source venv/bin/activate`.
3. Instalar dependencias con `pip install ..`.
4. Arrancar con `uvicorn app.main:app --reload`.
5. Abrir `http://127.0.0.1:8000`.

5.3. Instalación con Docker

1. Construir imagen: `docker build -t gestor-identidades .`
2. Ejecutar contenedor: `docker run -p 8000:8000 gestor-identidades`
3. Abrir `http://localhost:8000`

5.4. Despliegue en Render

El proyecto incluye `render.yaml` con:

- servicio web Docker;
- `healthCheckPath=/health`;
- PostgreSQL administrado;
- secretos generados para `SESSION_SECRET` y `DATABASE_ENCRYPTION_KEY`.

Producción En producción, `SESSION_SECRET` no puede quedarse con su valor por defecto. Si `ENVIRONMENT=production`, la aplicación falla intencionalmente si no se define un secreto seguro.

6. Ejemplos de Interfaz

Esta sección reúne únicamente los flujos de interfaz más útiles para entender cómo se usa visualmente la aplicación. No intenta cubrir todo el contenido de un manual de usuario completo.

6.1. Ejemplo 1: Iniciar sesión como usuario no criptográfico

Aplica para perfiles como OPERATIVO y VOLUNTARIO.

Pasos:

1. Escribir el correo o identificador.
2. Escribir la contraseña.
3. Hacer clic en Entrar.

Resultado esperado Si la cuenta está activa y la contraseña es correcta, el sistema redirige al portal del usuario.

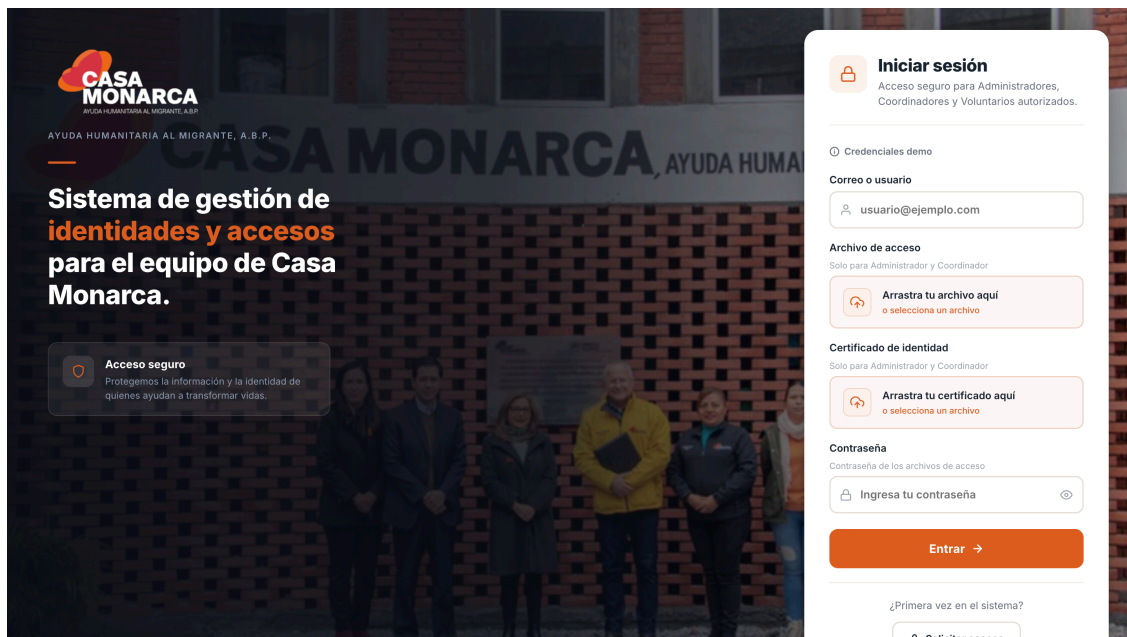


Figure 1: Página de inicio de sesión del sistema

6.2. Ejemplo 2: Iniciar sesión como usuario criptográfico

Aplica para ADMIN y COORDINADOR cuando ya tienen certificado emitido.

Pasos:

1. Subir private_key.pem.
2. Subir certificate.pem.
3. Capturar el identificador del usuario.
4. Capturar la contraseña que protege la llave privada.
5. Hacer clic en Entrar.

Qué valida el sistema El backend abre la llave privada, valida que el certificado pertenezca al usuario correcto, verifica emisor y vigencia, construye un challenge y confirma la firma con la llave pública del certificado.

Resultado esperado Si todas las validaciones criptográficas pasan, la sesión se crea correctamente.

6.3. Ejemplo 3: Acceso demo y usuarios disponibles

Cuando `SEED_DEMO_DATA=true`, existe un bypass demo para el administrador principal:

- identificador: admin
- contraseña: admin

Importante Este bypass solo aplica para el administrador demo y únicamente cuando la configuración lo permite. No sustituye el flujo criptográfico real de producción.

Usuario / identificador	Contraseña demo	Rol	Método de acceso vigente
admin	admin	Administrador	Bypass demo sin archivos criptográficos cuando la configuración lo permite.
operativo@demo.local	demo1234	Operativo	Acceso por contraseña.
voluntario@demo.local	demo1234	Voluntario	Acceso por contraseña.
coord.administracion@demo.local	demo1234	Coordinador	Requiere private_key.pem y certificate.pem si ya tiene certificado emitido.
coord.legal@demo.local	demo1234	Coordinador	Requiere private_key.pem y certificate.pem si ya tiene certificado emitido.
coord.psicosocial@demo.local	demo1234	Coordinador	Requiere private_key.pem y certificate.pem si ya tiene certificado emitido.
coord.humanitario@demo.local	demo1234	Coordinador	Requiere private_key.pem y certificate.pem si ya tiene certificado emitido.
coord.comunicacion@demo.local	demo1234	Coordinador	Requiere private_key.pem y certificate.pem si ya tiene certificado emitido.

Table 7: Usuarios demo y método de acceso vigente según la implementación actual

6.4. Ejemplo 4: Crear un usuario desde administración

Ruta implicada: `/ui/users`

Pasos:

1. Entrar al panel administrativo.
2. Capturar nombre completo, correo, rol, fecha de expiración opcional y secreto inicial.

3. Enviar el formulario.

Ejemplo:

Crear a María García con correo maria.garcia@casamonarca.org y rol COORDINADOR.

Resultado esperado La cuenta se crea en estado pending. Si el rol requiere criptografía, el sistema emite el certificado y el material criptográfico correspondiente.

Figure 2: Formulario de creación de usuario en el panel de administración

6.5. Ejemplo 5: Auto-registro externo

Ruta implicada: /register

Pasos:

1. Abrir Solicitar acceso.
2. Capturar nombre completo, correo y contraseña.
3. Confirmar la contraseña.
4. Enviar la solicitud.

Resultado esperado El sistema crea un usuario con rol VOLUNTARIO en estado pending. Un administrador debe activarlo para permitir el acceso.

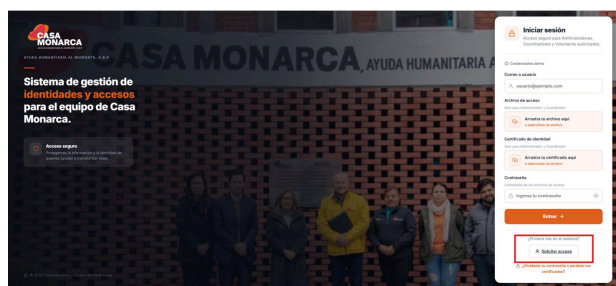


Figure 3: Creación de usuario externo

6.6. Ejemplo 6: Activar un usuario pendiente

Ruta implicada: `/ui/users/{user_id}/status`

Pasos:

1. Localizar al usuario pendiente.
2. Seleccionar el cambio de estado a active.
3. Si aplica, capturar `new_secret`.
4. Confirmar la acción.

Resultado esperado La cuenta pasa a estado `active` y el evento queda auditado. Si el rol es criptográfico, pueden emitirse o refrescarse credenciales.

The screenshot shows the user management interface for 'Maria Fernanda' (mariafernanda@gmail.com). The user is in the 'Voluntario' role and has a 'pending' status. The interface includes a search bar, filters for roles and states, and a list of users. Below the list, there are four tabs: CUENTA, VIGENCIA, ROL, and ACCESO. The CUENTA tab is active, showing a message 'La cuenta existe pero todavía no puede entrar.' and an 'Activar' button highlighted with a red box. Other buttons include 'Revocar de emergencia', 'Guardar vigencia', 'Guardar rol', and 'Gestionar'.

Figure 4: Activación de usuario pendiente

The left screenshot shows the user management interface for 'Maria Fernanda' (mariafernanda@gmail.com). The user is in the 'Coordinador' role and has an 'active' status. The interface includes a search bar, filters for roles and states, and a list of users. Below the list, there are four tabs: CUENTA, VIGENCIA, ROL, and ACCESO. The CUENTA tab is active, showing a message 'La cuenta existe pero todavía no puede entrar.' and an 'Activar' button highlighted with a red box. Other buttons include 'Revocar de emergencia', 'Guardar vigencia', 'Guardar rol', and 'Gestionar'.

The right screenshot shows the user management interface for 'Maria Fernanda' (mariafernanda@gmail.com). The user is in the 'Coordinador' role and has an 'active' status. The interface includes a search bar, filters for roles and states, and a list of users. Below the list, there are four tabs: CUENTA, VIGENCIA, ROL, and ACCESO. The ACCESO tab is active, showing a message 'La vigencia de este usuario está lista en su certificado de identidad no puede modificarse directamente. Para extender el acceso, revocar la cuenta o generar nuevas credenciales desde la sección Credenciales.' and a 'Descargar' button highlighted with a red box. Other buttons include 'Revocar de emergencia', 'Guardar vigencia', 'Guardar rol', and 'Gestionar'.

Figure 5: Descarga y entrega de credenciales para un usuario con rol criptográfico

6.7. Ejemplo 7: Revocar un usuario

Pasos:

1. Localizar al usuario activo.
2. Cambiar su estado a revoked.
3. Confirmar la operación.

Resultado esperado El usuario ya no puede iniciar sesión. Si tenía material criptográfico, su acceso queda invalidado y el evento se registra en auditoría.



Figure 6: Revocación de acceso de un usuario

6.8. Ejemplo 8: Cambiar el rol de un usuario

Ruta implicada: `/ui/users/{user_id}/role`

Pasos:

1. Seleccionar el usuario.
2. Elegir el nuevo rol.
3. Si el nuevo rol requiere credenciales criptográficas y antes no las requería, proporcionar `new_secret`.
4. Guardar cambios.

Resultado esperado El sistema actualiza el rol. Si el cambio migra a un rol criptográfico, emite nuevas credenciales para el usuario.

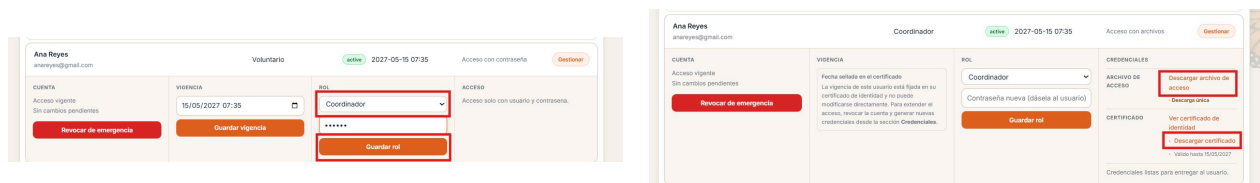


Figure 7: Cambio de rol y reemisión de credenciales

6.9. Ejemplo 9: Reactivar o restaurar acceso

Para reactivar un usuario revocado o actualizar credenciales:

1. Ubicar al usuario.
2. Cambiar el estado a `active`.
3. Proporcionar un nuevo secreto si el sistema lo requiere.

Caso típico Esta operación es útil cuando una cuenta fue revocada, cuando se necesita restablecer acceso o cuando hay que emitir nuevamente credenciales.

Figure 8: Reactivación de un usuario

6.10. Ejemplo 10: Descargar credenciales criptográficas

Rutas relacionadas:

- /ui/users/{user_id}/certificate.pem
- /ui/users/{user_id}/public-key.pem
- /ui/users/{user_id}/private-key.pem

Pasos:

1. Abrir la sección de credenciales del usuario.
2. Descargar el certificado.
3. Descargar la llave privada si está disponible.

Regla crítica La llave privada puede quedar marcada como entregada una sola vez. Si ya no está disponible, el administrador debe reemitir credenciales.

Figure 9: Descarga del certificado y del archivo de acceso

6.11. Ejemplo 11: Registrar un beneficiario

Ruta implicada: /ui/beneficiarios

Pasos:

1. Abrir la sección Beneficiarios.

2. Capturar nombre completo, país de origen, área y notas opcionales.
3. Enviar el formulario.

Áreas válidas:

- ADMINISTRACION
- LEGAL
- PSICOSOCIAL
- HUMANITARIO
- COMUNICACION

Resultado esperado El beneficiario se registra correctamente y queda visible en la sección operativa correspondiente.

The screenshot displays the 'Beneficiarios' management interface. At the top, there are summary statistics: 4 ACTIVOS, 10 TOTAL, 2 EN REVISIÓN, and 2 NUEVOS. A red box highlights the 'Registrar beneficiario' form, which contains the following fields:

- Nombre completo:** Julian Armenta
- País de origen:** Honduras
- Área de atención:** Humanitario
- Notas (opcional):** En trámite migratorio por motivos de seguridad

Below the form, a table lists existing beneficiaries:

Nombre	ID	Fecha	Área	Estado	Acciones
Maria Guadalupe Torres	1401/2018	14/01/2018	Psicosocial	Activo	Quitar, Eliminar
Carlos Enrique Fuentes	81/2018	07/05/2018	Legal	En revisión	Quitar, Eliminar
Aisha Balde	Guinea Conakry - 30/04/2018	30/04/2018	Humanitario	Nuevo	Quitar, Eliminar
Roberto Diaz Pérez					

Figure 10: Registro de un beneficiario

6.12. Ejemplo 12: Actualizar el estado de un beneficiario

Ruta implicada: `/ui/beneficiarios/{ben_id}/status`

Solo ADMIN y COORDINADOR pueden hacerlo.

Estados válidos:

- nuevo
- en_revision
- canalizado
- activo

Resultado esperado El estado cambia y queda persistido de inmediato.

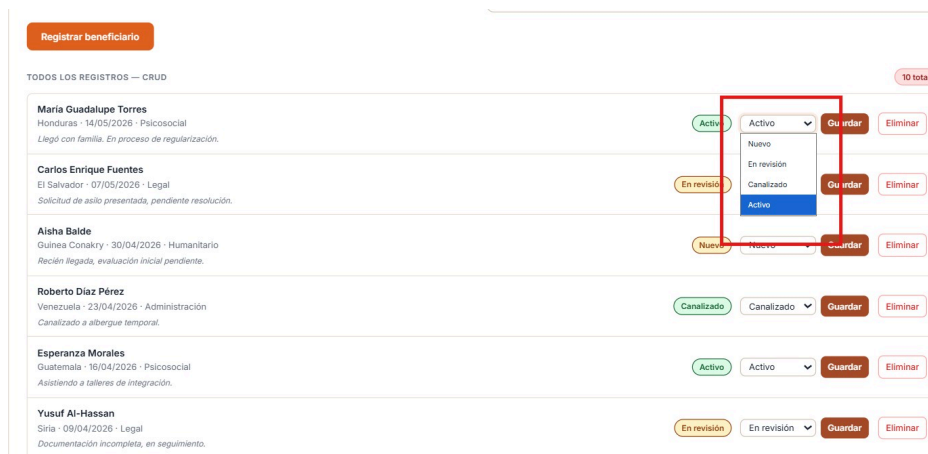


Figure 11: Actualización del estado de un beneficiario

6.13. Ejemplo 13: Exportar reportes PDF

Rutas disponibles:

- /ui/users/export.pdf
- /ui/audit-logs/export.pdf
- /ui/beneficiarios/export.pdf

Reporte	Acceso	Contenido
Usuarios	Administrador / Coordinador	Usuarios, roles, estados y fechas.
Auditoría	Administrador	Eventos, acciones, resultados, IP y user-agent.
Beneficiarios	Administrador / Coordinador	Nombre, país, área y estado.

Table 8: Reportes PDF exportables desde la aplicación

Integridad Los reportes PDF incluyen un hash SHA-256 calculado sobre los datos del reporte para apoyar la verificación de integridad.

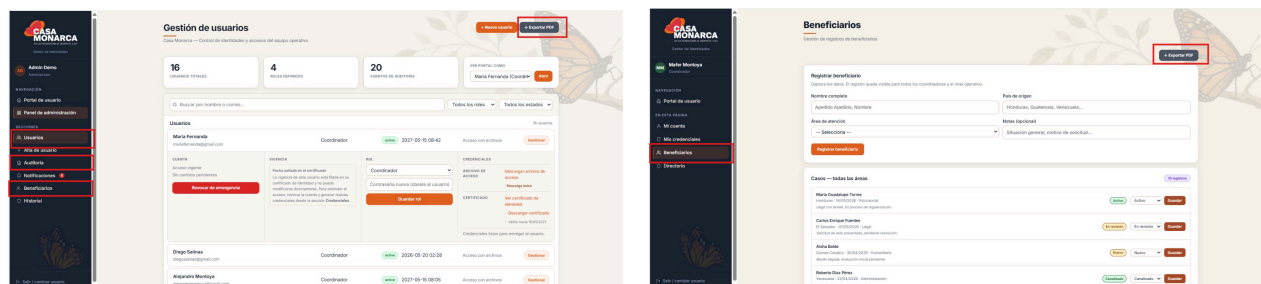


Figure 12: Vista de reportes para administrador y usuario

6.14. Ejemplo 14: Desbloquear una cuenta bloqueada

Ruta implicada: `/ui/users/{user_id}/unlock`

El sistema bloquea cuentas después de múltiples intentos fallidos consecutivos.

Pasos:

1. Ubicar la cuenta bloqueada.
2. Ejecutar la acción de desbloqueo.

Resultado esperado Se reinician `login_attempts` y `login_locked_until`, y la cuenta puede volver a autenticarse.

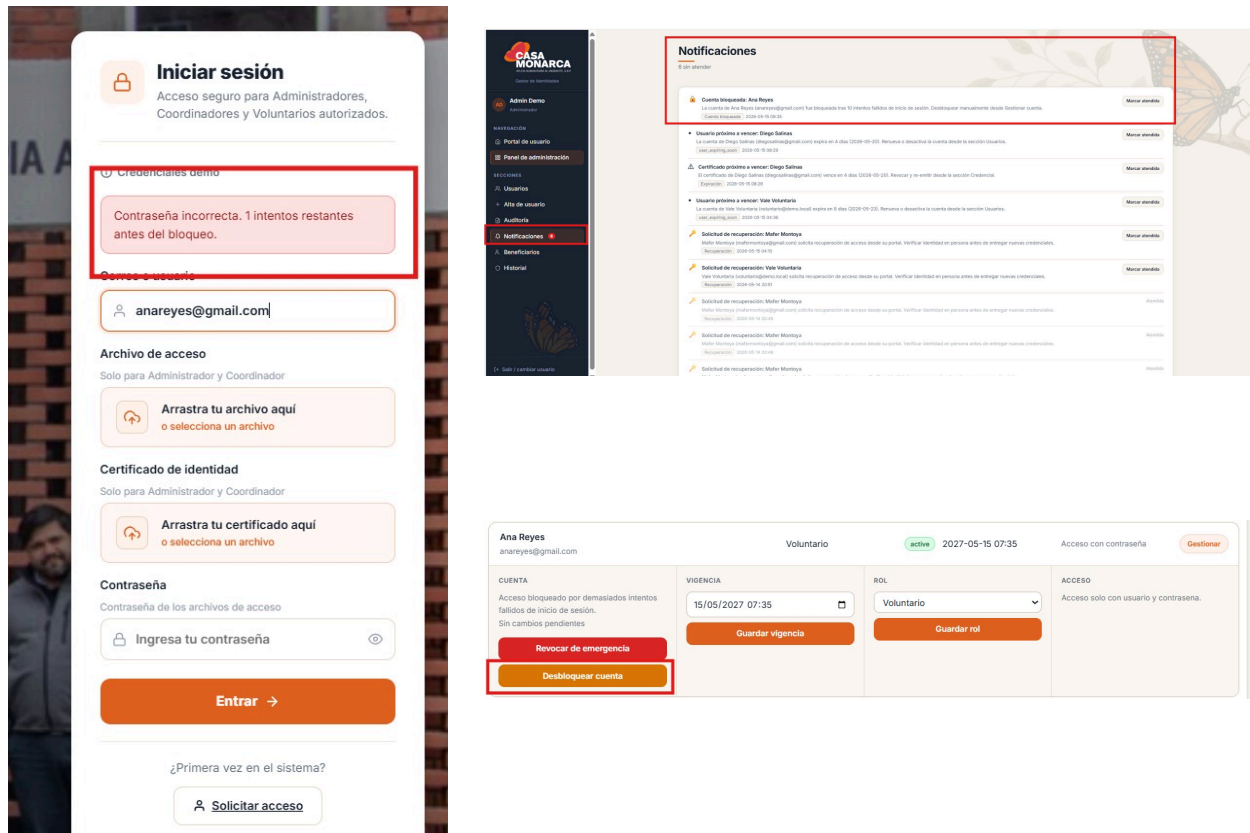


Figure 13: Desbloqueo de una cuenta bloqueada

6.15. Ejemplo 15: Activar el administrador espejo

Ruta implicada: `/ui/admin/recovery/activate-mirror`

Pasos:

1. Iniciar sesión como administrador principal activo.
2. Ir al módulo de recuperación.
3. Activar el espejo.

Procedimiento irreversible El administrador principal queda revocado y el administrador espejo pasa a ser el nuevo activo. Esta acción debe reservarse para emergencias reales.

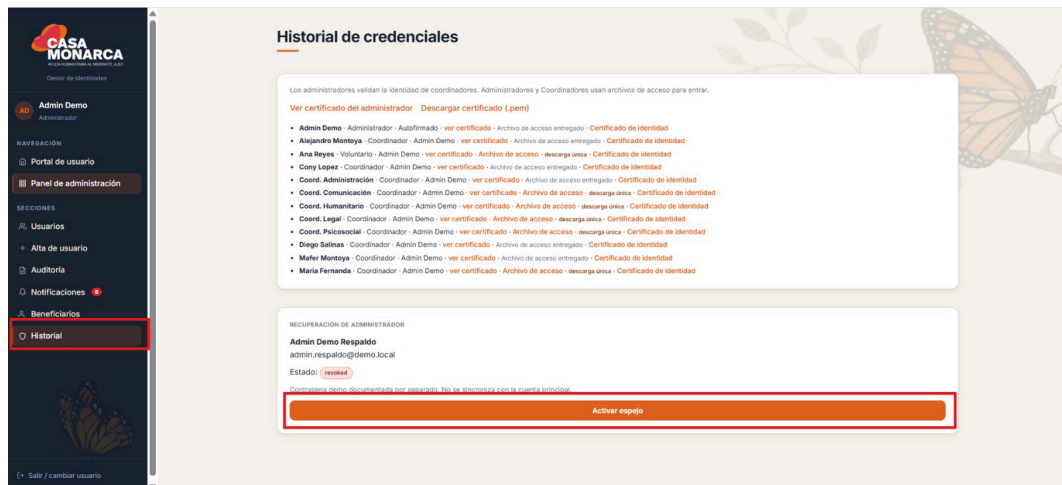
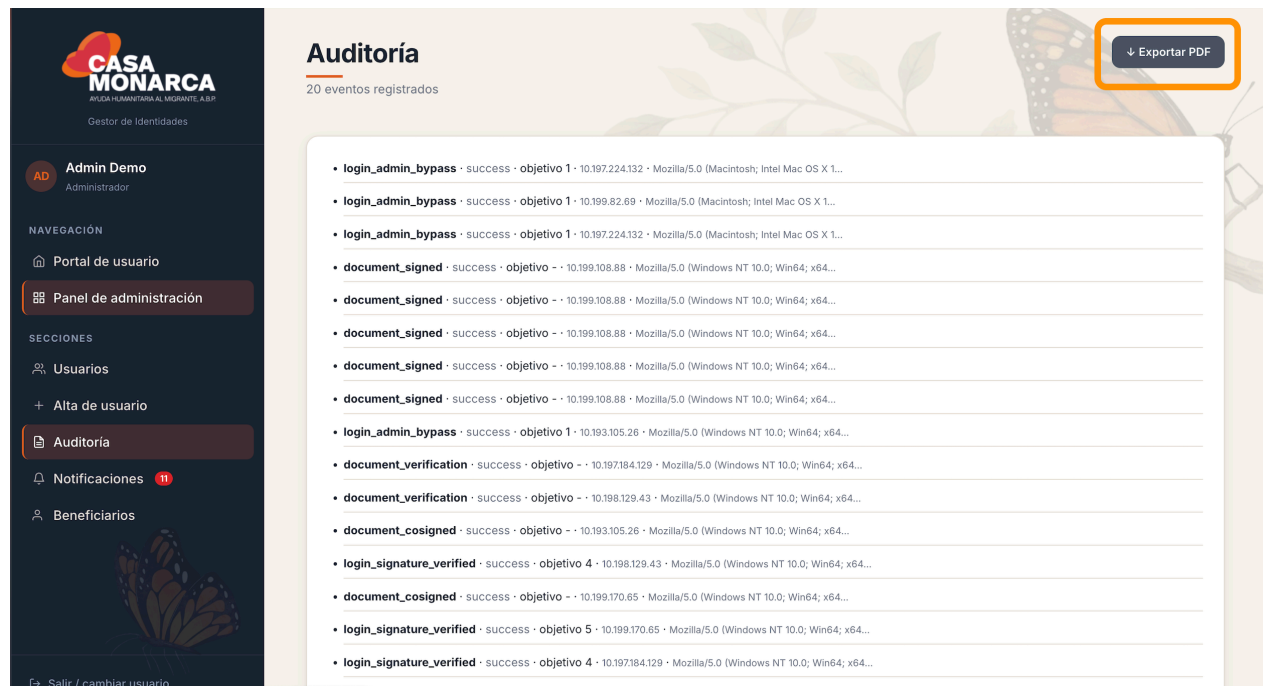


Figure 14: Activación del administrador espejo

6.16. Ejemplo 16: Consultar la auditoría

Ruta implicada: `/api/audit-logs` y vista administrativa.

La auditoría permite revisar:



- tipo de evento;
- acción;
- resultado;
- usuario actor;
- usuario objetivo;
- IP;
- user-agent;
- fecha y hora.

Resultado esperado El equipo administrativo puede reconstruir acciones sensibles del sistema y exportarlas a PDF para trazabilidad.

6.17. Ejemplo 17: Firmar un documento

Ruta implicada: /ui/documents/sign

Pasos:

1. Capturar título y tipo documental.
2. Seleccionar archivo permitido.
3. Definir vigencia opcional en días.
4. Enviar la solicitud de firma.

Parámetro	Valor / Regla
Tipos documentales	constancia, carta, reporte, oficio, otro
Extensiones	.pdf, .doc, .docx, .txt, .odt
Tamaño máximo	20 MB
Hash	SHA-256
Firma	RSA-PSS-SHA256

Table 9: Reglas funcionales del módulo documental

Resultado esperado Se genera un folio único y el documento queda firmado o, si requiere más firmantes, en estado pending_signatures.

CASA MONARCA
AYUDA HUMANITARIA AL MIGRANTE A.B.P.
Gestor de Identidades

Admin Demo
Administrador

NAVEGACIÓN

- Portal de usuario
- Panel de administración

EN ESTA PÁGINA

- Mi cuenta
- Mis credenciales
- Beneficiarios
- Firmar documentos**
- Verificar documento

Documentos
Firma digital de documentos y verificación de autenticidad

Firmar documento
Carga un documento para firmarlo digitalmente con tu certificado X.509. Se generará un folio único de verificación.

Proceso criptográfico: Se calcula el hash SHA-256 del documento, se firma con tu llave privada RSA (RSA-PSS-SHA256) y se verifica inmediatamente con tu certificado público. El documento original **no se almacena** en el servidor — solo el hash y la firma digital.

Título del documento
Constancia de atención, Carta institucional...

Tipo de documento
Buscar tipo...

Vigencia (días, opcional)
Dejar vacío para sin vencimiento

Requiere co-firmantes
No — solo yo firmo

Archivo a firmar
Formatos: PDF, DOC, DOCX, TXT, ODT — Máximo 20 MB

Arrastra tu documento aquí
o selecciona un archivo

Firmar documento

6.18. Ejemplo 18: Firma masiva de documentos

Ruta implicada: /ui/documents/batch-sign

Pasos:

1. Seleccionar un tipo documental.
2. Subir varios archivos.
3. Definir vigencia opcional.
4. Ejecutar firma masiva.

Límite actual El backend procesa hasta 20 archivos por lote.

Firma masiva (Batch Signing)
 Selecciona múltiples documentos para firmarlos todos con un solo clic. Ideal para constancias de atención del día.

Tipo de documento (aplica a todos) Vigencia (días, opcional)
 -- Selecciona -- Sin vencimiento

Archivos a firmar
 Selecciona hasta 20 archivos — PDF, DOC, DOCX, TXT, ODT

Arrastra tus documentos aquí
 o selecciona múltiples archivos

Firmar todos

Documentos firmados 7 documentos Exportar CSV

Buscar por folio, título o firmante...

Todos los tipos

Todos los estados

6.19. Ejemplo 20: Verificar un documento por folio

Rutas implicadas:

- /verificar
- /verificar/{folio}

Pasos:

1. Abrir el portal público.
2. Capturar el folio del documento.
3. Ejecutar verificación.

Resultado esperado El sistema devuelve un resultado como valid, revoked, expired, not_found o invalid_signature.

CASA MONARCA
AYUDA HUMANITARIA AL MIGRANTE, A.S.P.
Gestor de Identidades

Admin Demo
Administrador

NAVEGACIÓN

- Portal de usuario
- Panel de administración

EN ESTA PÁGINA

- Mi cuenta
- Mis credenciales
- Beneficiarios
- Firmar documentos
- Verificar documento**

[←] Salir / cambiar usuario

Verificación de documentos

Comprueba si un documento fue emitido oficialmente por Casa Monarca

Verificar por folio

Ingresa el folio que aparece en el documento.

Folio del documento

CM-2026-DOC-XXXXXXXX

Verificar folio

Verificar por archivo

Carga el documento recibido para comparar su contenido con el hash original registrado.

El sistema calcula el hash SHA-256 del archivo y lo compara con el hash registrado al momento de la firma digital. Si coincide, el documento no fue alterado.

Archivo a verificar

Arrastra tu documento aquí
o selecciona un archivo

Verificar archivo

6.20. Ejemplo 21: Verificar un documento por archivo

Ruta implicada: /verificar/archivo

Pasos:

1. Abrir el portal público.
2. Subir el archivo recibido.
3. Si se desea, proporcionar también el folio.
4. Ejecutar verificación.

Qué comprueba esta ruta El backend recalcula el SHA-256 del archivo subido, compara contra el hash persistido y valida además la firma criptográfica del documento.

7. Referencias Técnicas

FastAPI Documentation. Disponible en: <https://fastapi.tiangolo.com/>

SQLAlchemy Documentation. Disponible en: <https://docs.sqlalchemy.org/>

Cryptography Documentation. Disponible en: <https://cryptography.io/>

itsdangerous Documentation. Disponible en: <https://itsdangerous.palletsprojects.com/>

Docker Documentation. Disponible en: <https://docs.docker.com/>

Render Documentation. Disponible en: <https://render.com/docs>

RFC 5280 — Internet X.509 Public Key Infrastructure Certificate and CRL Profile.

RFC 8017 — PKCS 1: RSA Cryptography Specifications Version 2.2.

NIST SP 800-63B — Digital Identity Guidelines: Authentication and Lifecycle Management.